

APPLIED AGENTIC AI FOUNDATION PROGRAMME

Beginner Track · Windows · Terminal-first

MONTH 2 — LAB GUIDE

Build a Quiz Master Bot

A bot that runs quizzes, keeps score, and remembers players — one very small step at a time.

Builds directly on Month 1. If you can write, run, and debug a simple Python program, you're ready.

What you will build this month

- A quiz bot that asks a series of questions and checks each answer.
- It keeps a running score and tells the player how they did at the end.
- It saves every player's score to a file, so a leaderboard survives between runs.
- It supports categories and difficulty levels — and ships to GitHub as v1.0.

Before you start

- Open your existing **chatbot** work folder in VS Code, or make a new folder called **quizbot** the same way you did in Month 1.
- You'll create one main file this month called **quiz.py**.
- Same habit as Month 1: type the code (don't paste), save with **Ctrl + S**, run with the ► button, check the green box.

Month 2 at a glance

Last month your bot replied to messages. This month it runs a structured activity — a quiz — which means it needs to store many questions, track a score, and remember results. Those three needs teach you the most useful tools in all of programming: lists, dictionaries, and files.

Week	Focus	What you ship
Week 1	Lists, dictionaries & storing questions	A quiz that asks stored questions (v0.1)
Week 2	Scoring, loops & many questions	A full scored quiz that loops to the end (v0.2)
Week 3	Saving results to a file & high scores	A quiz that remembers scores between runs (v0.3)
Week 4	Categories, difficulty & ship	A polished quiz agent with a leaderboard (v1.0)

New this month: lists (ordered collections), dictionaries (labelled data), the for loop (do something for every item), and reading/writing files (so data survives after the program closes).

Week 1 — Lists, dictionaries & storing questions

Goal of the week: learn how to store many pieces of data neatly, then use that to hold your quiz questions and ask them one by one.

Day 1 — Lists: storing many things

A quiz has many questions. Storing each in its own variable would be a nightmare. A list holds many values in one place, in order.

Task 1.1 — Make your first list

1. Make a new file `day1.py` and type:

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
print(fruits[0])
print(fruits[1])
```

2. Save and run.

✓ What you should see

```
['apple', 'banana', 'cherry']
apple
banana
```

Counting starts at 0. `fruits[0]` is the first item, `fruits[1]` the second. This trips up every beginner once — now you know.

Task 1.2 — Add and count items

3. Below your list, add:

```
fruits.append("orange")
print(fruits)
print("Number of fruits:", len(fruits))
```

4. Save and run. Notice orange is now on the end, and `len()` counts the items.

✓ What you should see

```
['apple', 'banana', 'cherry', 'orange']
Number of fruits: 4
```

🚩 **Checkpoint:** You can create a list, read items by their position, add items, and count them.

Day 2 — The for loop: do something for every item

To ask every question in a quiz, you need to go through a list one item at a time. That's exactly what a for loop does.

Task 2.1 — Loop through a list

5. New file `day2.py`:

```
questions = ["What is 2 + 2?", "What colour is the sky?"]
```

```
for q in questions:
    print(q)
```

6. Save and run. Both questions print, each on its own line.

✓ **What you should see**

What is 2 + 2?

What colour is the sky?

Read it as a sentence: “for each q in questions, print q.” The word **q** is just a temporary name for the current item — you can call it anything.

Task 2.2 — Ask each question and collect the answer

7. Change the loop so it asks each question instead of just printing it:

```
questions = ["What is 2 + 2?", "What colour is the sky?"]
for q in questions:
    answer = input(q + " ")
    print("You said:", answer)
```

8. Save and run. It asks both questions, waiting for you to type each answer.

🚩 **Checkpoint:** You can loop through a list of questions and collect an answer for each — the heart of a quiz.

Day 3 — Dictionaries: pairing questions with answers

A quiz needs to know the correct answer to each question. A dictionary stores data in labelled pairs — perfect for pairing a question with its answer.

Task 3.1 — Your first dictionary

9. New file **day3.py**:

```
capital = {"France": "Paris", "Japan": "Tokyo"}
print(capital["France"])
print(capital["Japan"])
```

10. Save and run.

✓ **What you should see**

Paris

Tokyo

A dictionary uses curly braces **{ }**. Each entry is a **key: value** pair. You look up a value by its key, like **capital["France"]**.

Task 3.2 — Store one question as a dictionary

11. A neat way to store a quiz question with its answer:

```
q1 = {"question": "What is 2 + 2?", "answer": "4"}
print(q1["question"])
print("Correct answer is:", q1["answer"])
```

12. Save and run. You now have a tidy structure for a single question.

 **Checkpoint:** You can store labelled data in a dictionary and look values up by key.

Day 4 — A list of dictionaries: your question bank

Now combine both ideas: a list (many questions) where each item is a dictionary (a question + its answer). This is your whole quiz in one neat structure.

Task 4.1 — Build the question bank

13. New file `day4.py`:

```
questions = [
    {"question": "What is 2 + 2?", "answer": "4"},
    {"question": "Capital of Japan?", "answer": "Tokyo"},
    {"question": "How many days in a week?", "answer": "7"}
]

print("First question:", questions[0]["question"])
print("Its answer:", questions[0]["answer"])
```

14. Save and run. Read the code slowly — you're reaching into a list, then into a dictionary.

✓ What you should see

```
First question: What is 2 + 2?
Its answer: 4
```

`questions[0]` gives the first dictionary; adding `["answer"]` then reaches inside it. This “list of dictionaries” pattern is used everywhere in real programs.

 **Checkpoint:** You have a proper question bank: a list of question-and-answer dictionaries.

Day 5 — Loop the question bank & check answers

Put it all together: loop through the question bank, ask each question, and check whether the user's answer matches the stored answer.

Task 5.1 — Ask and check

15. New file `day5.py` — reuse the `questions` bank from Day 4, then add:

```
for item in questions:
    user_answer = input(item["question"] + " ")
    if user_answer == item["answer"]:
        print("Correct!")
    else:
        print("Wrong. The answer was", item["answer"])
```

16. Save and run. Answer each question and watch it tell you right or wrong.

✓ What you should see

```
What is 2 + 2? 4
Correct!
```

Capital of Japan? Osaka
Wrong. The answer was Tokyo

🏁 **Checkpoint:** Your program asks every question and checks each answer. That's a working quiz core!

Days 6-7 — Fix, combine & ship v0.1

- Day 6: Review errors (same three from Month 1) plus a new common one — `KeyError`, which means you asked a dictionary for a key that doesn't exist. Fix by checking your spelling of keys like `"answer"`.
- Day 7: Create your main file `quiz.py`. Add a welcome message, the question bank, and the ask-and-check loop. Run a full quiz start to finish. This is v0.1.

🏁 **Checkpoint:** 🎉 Week 1 done. You have a quiz that asks stored questions and checks answers. Next week: scoring.

Week 2 — Scoring, loops & many questions

Goal of the week: track how many the player gets right, show a final score, and make the quiz easy to grow with more questions.

Day 8 — Keep a score

A score is just a number that starts at zero and goes up by one each time the player is correct.

Task 8.1 — Add a score counter

17. In `quiz.py`, create the score before the loop, and add to it on a correct answer:

```
score = 0
for item in questions:
    user_answer = input(item["question"] + " ")
    if user_answer == item["answer"]:
        print("Correct!")
        score = score + 1
    else:
        print("Wrong. The answer was", item["answer"])
```

18. Save and run a full quiz.

`score = score + 1` means “take the current score and make it one bigger.” You can also write it the short way: `score += 1`.

 **Checkpoint:** Your quiz counts correct answers as the player goes.

Day 9 — Show the final score

Task 9.1 — Print the result at the end

19. After the loop finishes (same indentation as `score = 0`, i.e. NOT indented under the `for`), add:

```
print("Quiz over! You scored", score, "out of", len(questions))
```

20. Save and run. At the end you now get a clear score summary.

✓ **What you should see**

Quiz over! You scored 2 out of 3

Watch your indentation: this line must line up with the `for`, not be inside it — otherwise it prints after every question instead of once at the end.

 **Checkpoint:** Your quiz shows a final score out of the total.

Day 10 — A friendlier result message

Task 10.1 — React to how they did

21. Use `if/elif/else` on the score to give encouraging feedback:

```
if score == len(questions):
    print("Perfect score! Amazing!")
```

```
elif score >= len(questions) / 2:  
    print("Good job!")  
else:  
    print("Keep practising – you will get there!")
```

22. Save and run a few times, answering well and badly, to see each message.

 **Checkpoint:** Your quiz responds warmly based on performance.

Day 11 — Make answers forgiving

Right now “Tokyo” works but “tokyo” or “ Tokyo ” fail. Reuse Month 1’s trick, `.lower()`, plus `.strip()` to remove stray spaces.

Task 11.1 — Clean the user's answer

23. Change the answer line to clean the input before comparing:

```
user_answer = input(item["question"] + " ")  
user_answer = user_answer.lower().strip()  
if user_answer == item["answer"].lower():  
    print("Correct!")
```

24. Save and run. Now capitalisation and extra spaces no longer cause wrong marks.

`.strip()` removes spaces from the start and end. Chaining `.lower().strip()` cleans the text in one go.

 **Checkpoint:** Your quiz fairly accepts answers regardless of case or spacing.

Days 12-14 — Grow & ship v0.2

- Day 12: Add at least 8 questions to your bank. Notice you didn't change any logic — that's the power of the list-of-dictionaries design.
- Day 13: Number the questions as they appear (e.g. “Question 1 of 8”) using a counter that goes up each loop.
- Day 14: Ship v0.2 — run it end to end, jot down what it does, and have a friend take your quiz.

 **Checkpoint:** End of Week 2: a complete, scored, forgiving quiz with a friendly result and many questions.

Week 3 — Saving results to a file

Goal of the week: right now, scores vanish when the program closes. This week you save them to a file so a leaderboard survives between runs — your first taste of data that persists.

Day 15 — Write text to a file

Task 15.1 — Create and write a file

25. New file `day15.py`:

```
file = open("scores.txt", "w")
file.write("Sam scored 3\n")
file.close()
print("Saved!")
```

26. Save and run. Then look in your folder in VS Code — a new file `scores.txt` has appeared. Click it to see the line inside.

✓ What you should see

Saved!
(and `scores.txt` now contains: Sam scored 3)

`open(..., "w")` opens a file for writing. `\n` means “new line”. Always `.close()` a file when done so it saves properly.

Note: `"w"` (write) erases the file first each time. To add without erasing, use `"a"` (append) — you'll use that next.

Day 16 — Append so you don't lose old scores

Task 16.1 — Add a line without erasing

27. Try appending three times — run this file three times and watch the file grow:

```
file = open("scores.txt", "a")
file.write("A new score line\n")
file.close()
```

28. Open `scores.txt` after each run. Old lines stay; new ones are added below.

 **Checkpoint:** You can create files, write to them, and append without losing existing data.

Day 17 — Save the real quiz score

Task 17.1 — Ask the player's name and save their result

29. In `quiz.py`, ask for a name at the start:

```
player = input("What is your name? ")
```

30. At the very end (after showing the score), append the result to the file:

```
file = open("scores.txt", "a")
file.write(player + " scored " + str(score) + "\n")
```

```
file.close()
print("Your score was saved!")
```

31. Save and run the full quiz. Then open `scores.txt` to see your result recorded.

`str(score)` turns the number into text so it can join the sentence. Text and numbers can't be added together directly — this converts it.

🚩 **Checkpoint:** Every completed quiz now records the player and their score to a file.

Days 18–21 — Read scores, show a leaderboard & ship v0.3

- Day 18: Read the file back with `open("scores.txt", "r")` and `.readlines()`, then loop through and print every past score.
- Day 19: Add a “View past scores” option at the start so players can see the leaderboard before playing.
- Day 20: Handle the very first run gracefully — if `scores.txt` doesn't exist yet, show “No scores yet!” instead of crashing (a gentle intro to `try/except`).
- Day 21: Commit and push to GitHub (same steps as Month 1). This is v0.3.

🚩 **Checkpoint:** Your quiz remembers scores across runs and shows a leaderboard — real persistent data.

Week 4 — Categories, difficulty & ship v1.0

Goal of the week: make the quiz feel like a real product — let players pick a category and difficulty, then polish and ship version 1.0.

Day 22 — Organise questions by category

Task 22.1 — Group questions

32. Use a dictionary where each key is a category name and each value is a list of questions:

```
quiz_bank = {
    "Maths": [
        {"question": "2 + 2?", "answer": "4"},
        {"question": "10 - 3?", "answer": "7"}
    ],
    "Geography": [
        {"question": "Capital of Japan?", "answer": "Tokyo"}
    ]
}
print("Categories available:")
for name in quiz_bank:
    print("-", name)
```

33. Save and run. It lists your categories. Looping a dictionary gives you its keys.

 **Checkpoint:** Your questions are organised into categories the player can choose from.

Day 23 — Let the player choose a category

- Ask the player to type a category name, then run the quiz using `quiz_bank[chosen_category]` as the question list.
- Handle a typo: if the category isn't found, show the list again and ask once more.

Day 24 — Add difficulty or a timer feel

- Add an “easy/hard” choice that changes how many questions are asked, or award more points for harder questions.

Day 25 — Make it feel polished

- Add clear section lines, a friendly intro screen, and a satisfying end screen showing score, category, and where they rank on the leaderboard.

Days 26–30 — Final polish & ship v1.0

- Day 26: Test everything. Try wrong inputs, empty answers, and unknown categories. Fix what breaks.
- Day 27: Write a `README.md` explaining how to play and what features it has.
- Day 28: Record a short demo video of someone taking your quiz.
- Day 29: Final commit and push (`git add .` → `git commit -m "v1.0"` → `git push`).
- Day 30: Show it off. You built a quiz game that stores data and keeps a leaderboard.

 Month 2 complete

You leveled up from simple replies to a structured application that manages many questions, tracks scores, and saves data to a file that survives between runs. Lists, dictionaries, loops, and files are the workhorses of nearly every program you'll ever write — and now they're yours. Month 3 uses them to give your next agent “eyes for data”.